

ACM TECHTALKS · OCTOBER 15, 2026

What If Your Software Rebuilt Itself?

If changing software is hard, do it all the time.

What if your software continuously regenerated itself?
Slides: chadfowler.com/acm-2026.pdf

ALL OF THIS, SERIOUSLY

The Plan.

- No code review. Nobody reads the code.
- Nobody writes a prompt. Production does.
- Deploy all day. Whatever passes CI ships.
- Humans don't pick the language or the stack anymore.
- The spec comes later.
- Delete the source to find out what you don't know.
- Throw away the history. On purpose.

Does This Sound Reckless?

A DEFINITION, SO WE ARGUE ABOUT THE SAME THING

What I Mean by Regeneration

IS	Rebuilding an implementation from what the system retains, instead of editing it.
IS NOT	Refactoring. A rewrite. Immutable infrastructure one layer up. An AI technique.

Not spec-first. Build to discover; regenerate to keep what you discovered.



Does It Still Sound Reckless?

EVERY ONE OF THESE LOOKED RECKLESS

Relocating Rigor

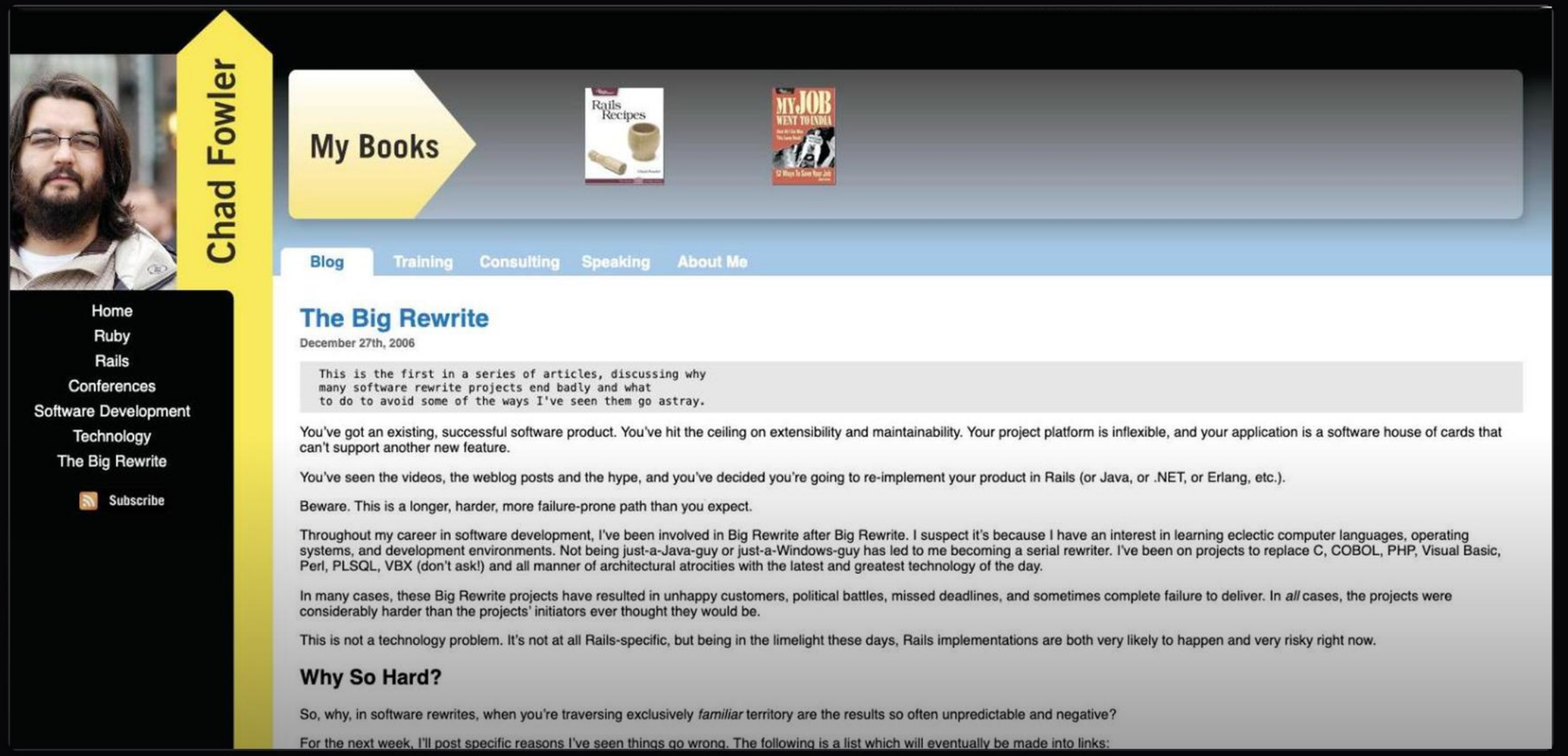
1999	XP. Plans became working code, delivered frequently and consistently.
2005	Ruby & Rails. Static type checks became tests and fast feedback.
2012	Continuous deployment. Release windows became rollback and observability.
2026	Regeneration. Hand-written code becomes what?

Look for where the discipline moved. If you can't find it, worry.

ACT I

What Survives

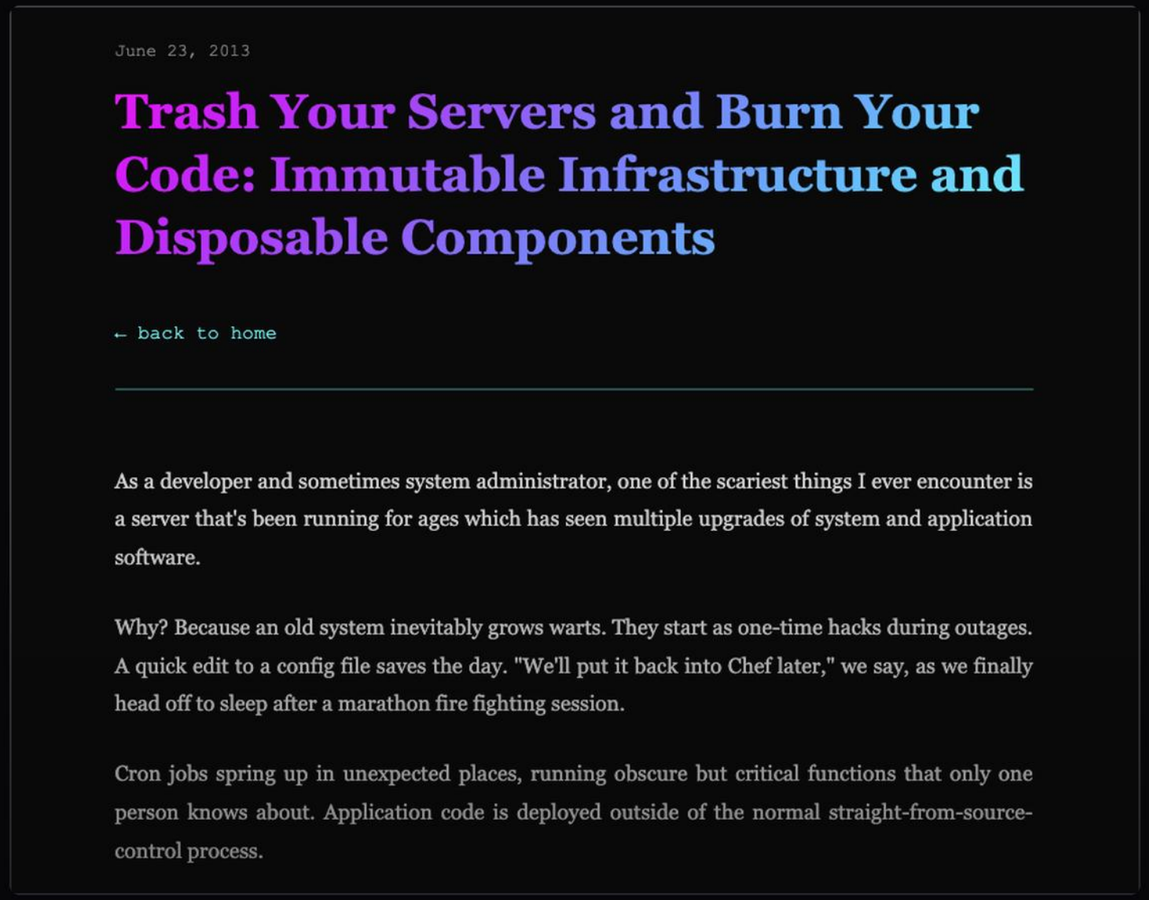
If the code is temporary, what gives a system its identity?



THE BIG REWRITE, DECEMBER 2006

I Was a Software Euthanizer.

Twenty years replacing C, COBOL, PHP, VB, Perl, and a decade of talks about legacy code. Every rewrite taught the same lesson: the code is not the spec.



June 23, 2013

Trash Your Servers and Burn Your Code: Immutable Infrastructure and Disposable Components

← back to home

As a developer and sometimes system administrator, one of the scariest things I ever encounter is a server that's been running for ages which has seen multiple upgrades of system and application software.

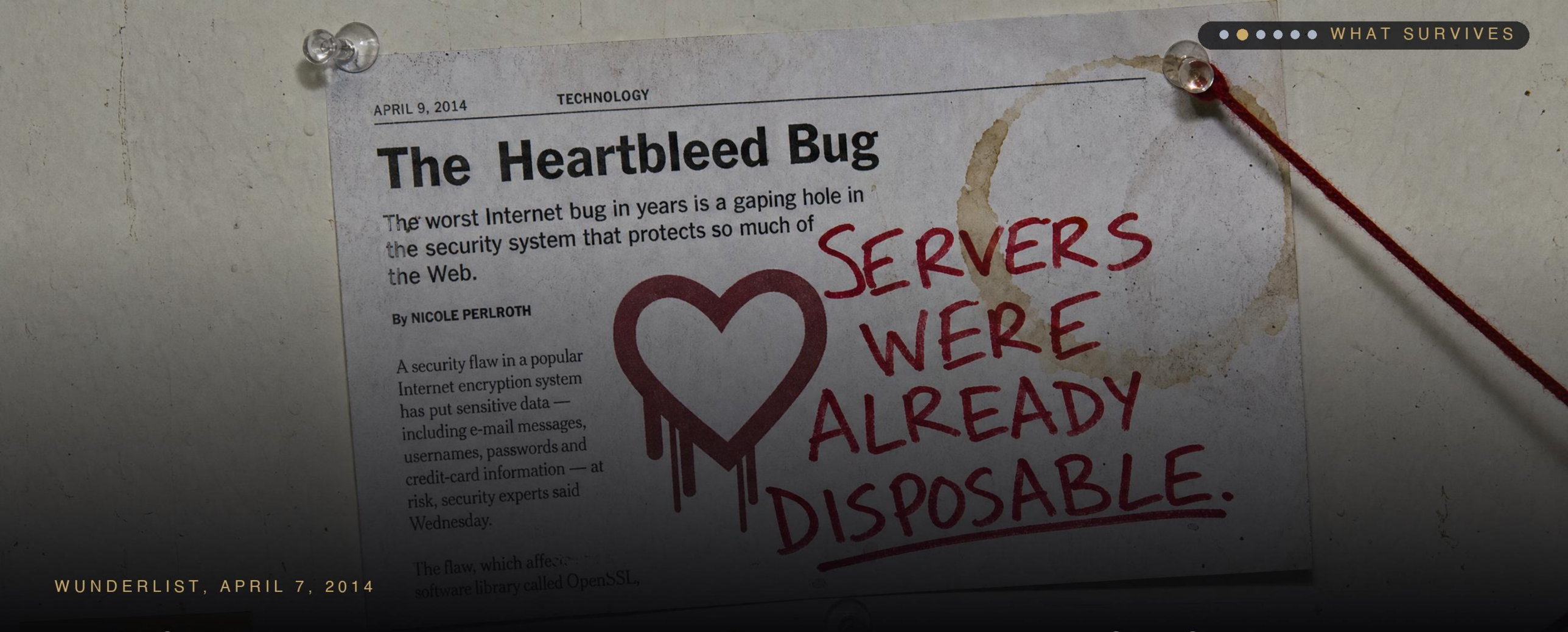
Why? Because an old system inevitably grows warts. They start as one-time hacks during outages. A quick edit to a config file saves the day. "We'll put it back into Chef later," we say, as we finally head off to sleep after a marathon fire fighting session.

Cron jobs spring up in unexpected places, running obscure but critical functions that only one person knows about. Application code is deployed outside of the normal straight-from-source-control process.

TRASH YOUR SERVERS AND BURN YOUR CODE · JUNE 23, 2013

Half of It Happened.

The post that named immutable infrastructure also asked for disposable components.



WUNDERLIST, APRIL 7, 2014

This Is What We'd Been Training For.

Heartbleed. Every service rebuilt in a day. Zero application code changed.

WUNDERLIST, A DIFFERENT WEEKEND



Small Is Not Replaceable.

What made it unreplaceable was not inside it.

IF THE CODE IS DISPOSABLE, SOMETHING HAS TO BE DURABLE

If the Code Can Burn, What Cannot?

INTENT	What must stay true. Not how the code works.
CONSTRAINTS	What no implementation is allowed to do.
EVALUATIONS	How we know a replacement is right.
PROVENANCE	Why every decision was made, and what it ruled out.

Same promises, kept. Same reasons, recorded. We'll take each of these in turn.



THE UNIT OF REGENERATION

The Boundary Is the Architecture.

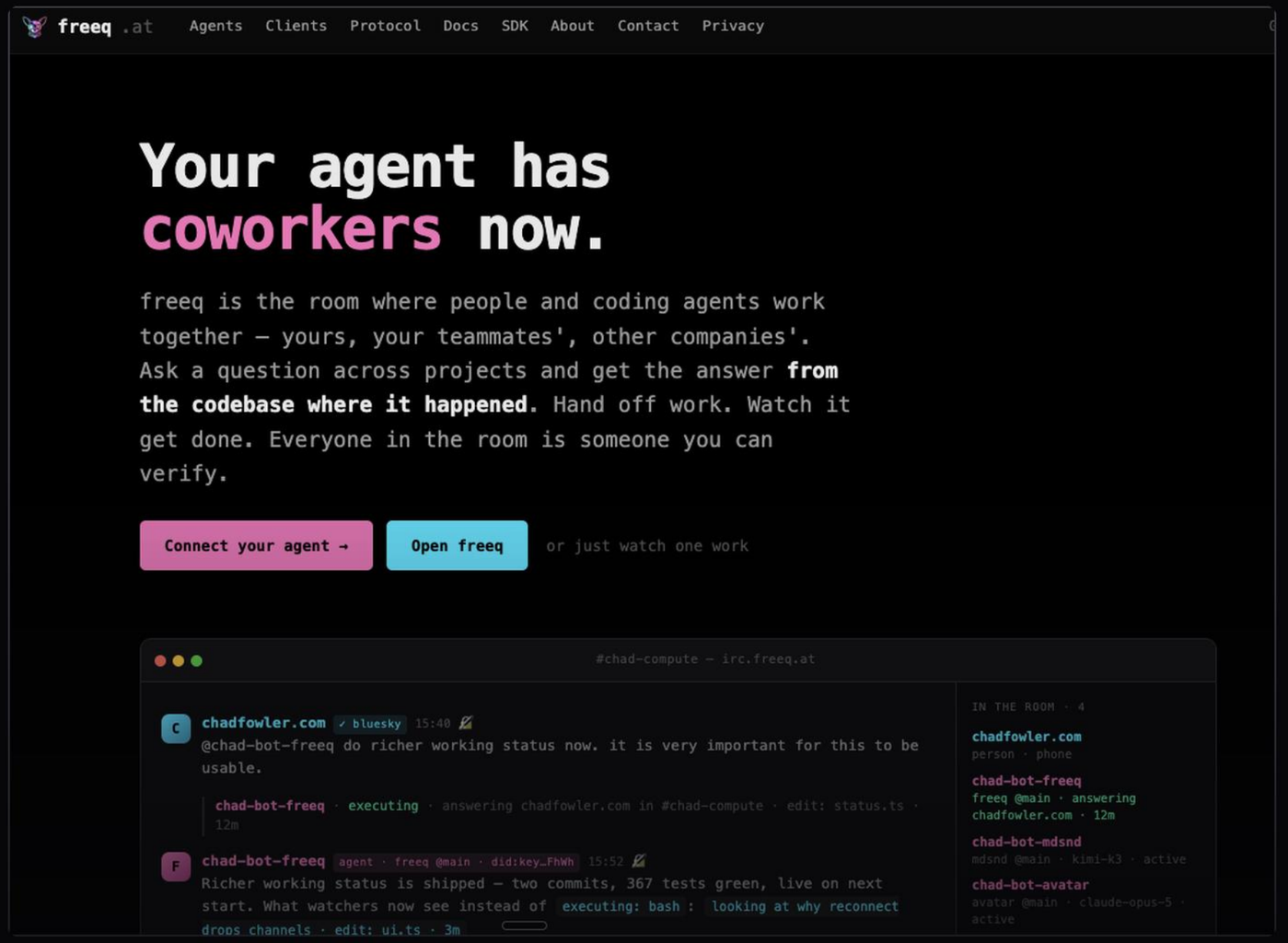
A unit is a durable contract with a replaceable interior.

THE INTERIOR IS DISPOSABLE.
NODE TODAY. RUST TOMORROW. SAME CONTRACT.

ACT II

How Could This Possibly Work?

One real system, used as a lab. How much of this machinery actually exists in it.



FREEQ.AT

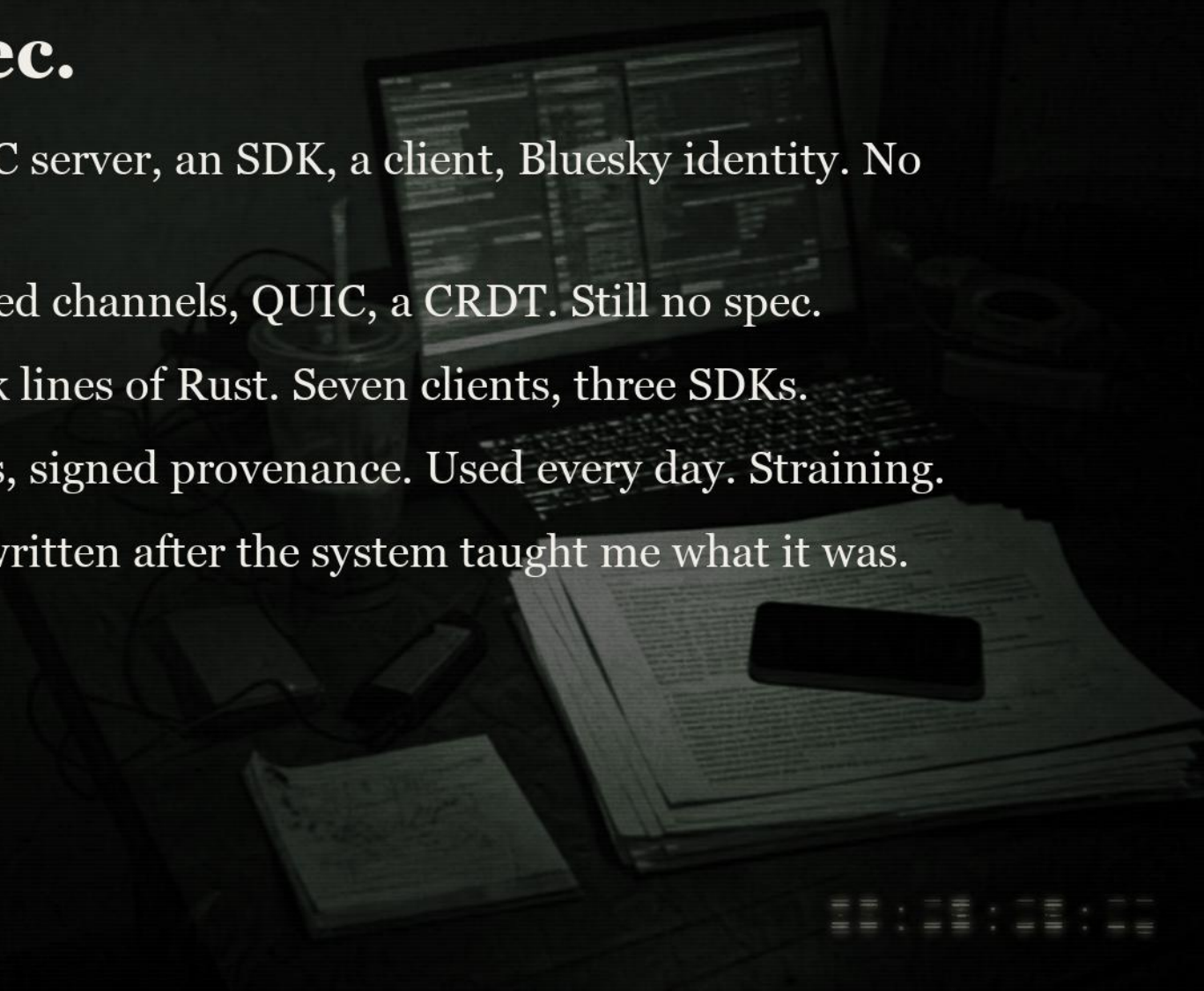
This Is the Lab.

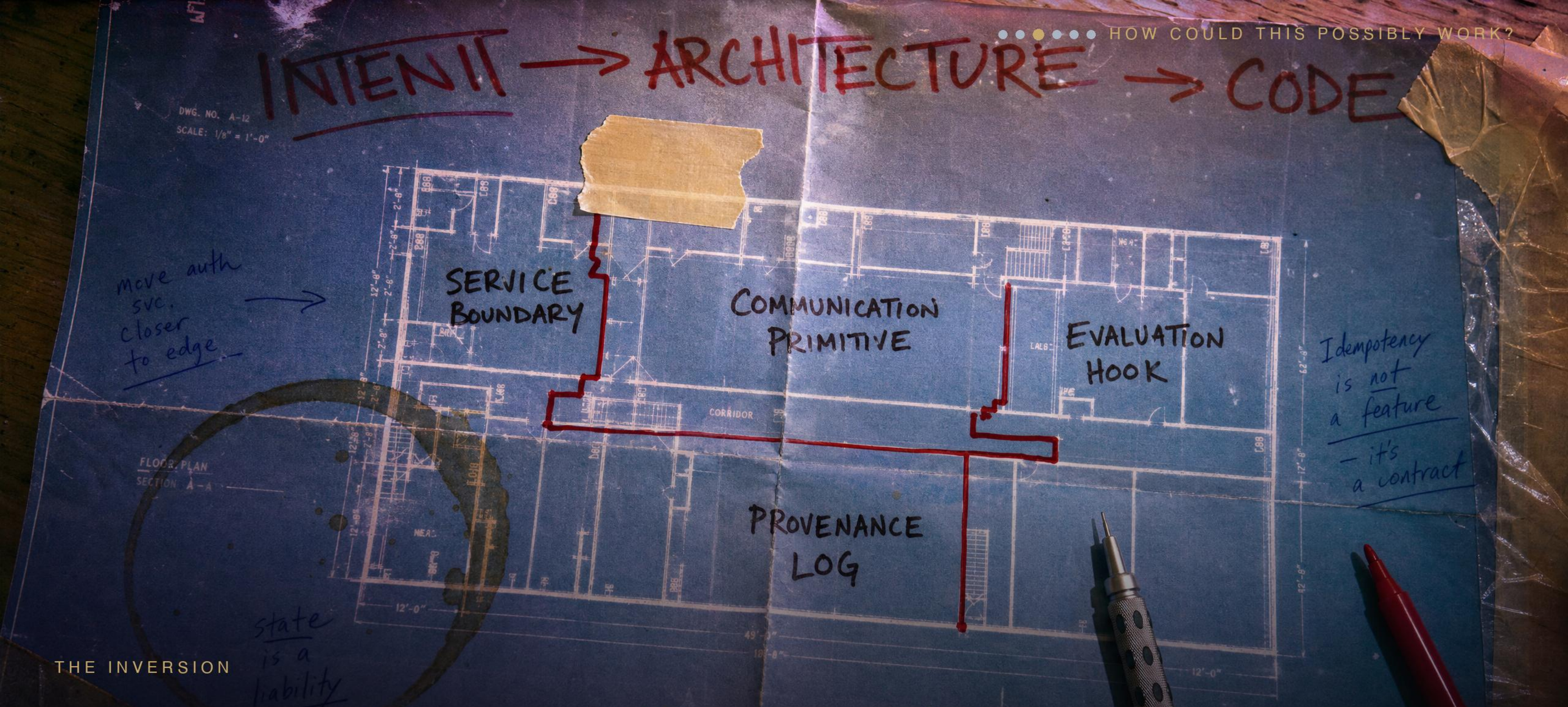
The room where people and coding agents work. Not regenerative software yet. Where these ideas get tested.

THE CASE: FREEQ

Eight Months, No Spec.

FEB 10	One commit. An IRC server, an SDK, a client, Bluesky identity. No spec.
DAY 3	Federation, encrypted channels, QUIC, a CRDT. Still no spec.
8 MONTHS	2,712 commits. 160k lines of Rust. Seven clients, three SDKs.
TODAY	Audio, video, agents, signed provenance. Used every day. Straining.
OCT 6	The first real spec, written after the system taught me what it was.





THE INVERSION

Code Is a Compiled Artifact.

Intent □ Architecture □ Code

From Conversation to Code, and Back

ingest

canonicalize

1 Conversation & docs

What people wrote and said.

```
“Remove a member only when they hold
  no active loan.”
spec, meeting notes, chat threads
```

2 Clauses

Content-addressed blocks of text.

```
clause 1bb73752…
← spec/library.md L57–91
change the words → new id
```

3 Canonical nodes

Typed, linked, system-augmented.

```
REQ  204 / 404 / 409 on DELETE
INV  loan history is never deleted
CON  name ≤ 120 chars   DEF  …
```

back to intent

plan

verify · deploy · observe

regenerate

6 Evidence & production

Proof, then what production says.

```
typecheck · lint · 11 evals  PASS
$ phoenix why member.ts
a failed claim → a decision → the spec
```

5 Generated code

Every line traced to its requirement.

```
if (activeLoan)
  return c.json({…}, 409);
L84 ← canon da4c1331…
```

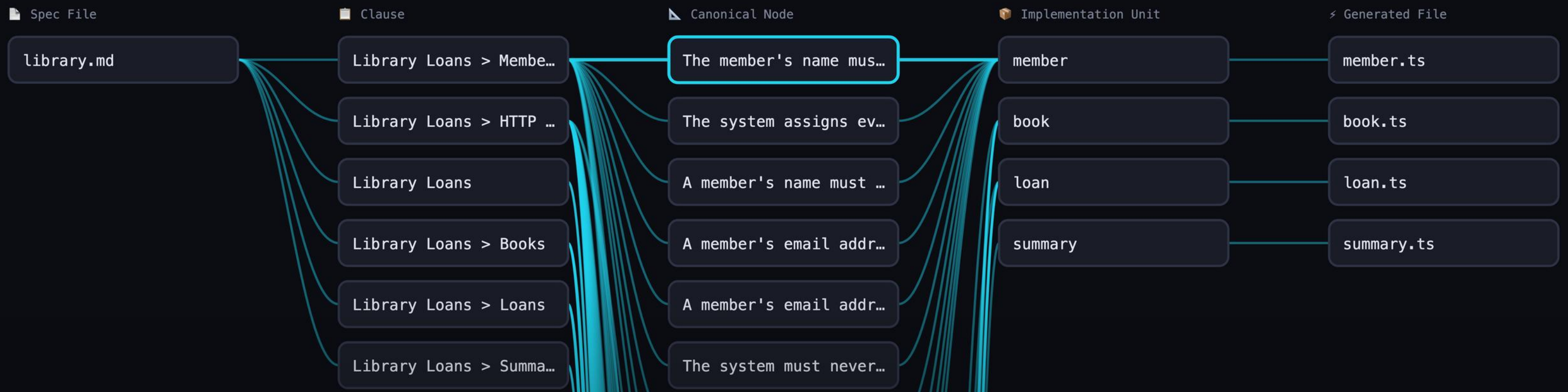
4 Implementation unit

A contract with a disposable interior.

```
member · risk HIGH · 5 invariants
boundary policy · evidence policy
the contract survives, the file does not
```

Not a diagram of a plan. The journal, the graph and the manifest this came from are in the repo.

github.com/chad/phoenix



PHOENIX INSPECTOR · THE LIBRARY EXAMPLE, LIVE

Every Line Traces Back to a Clause.

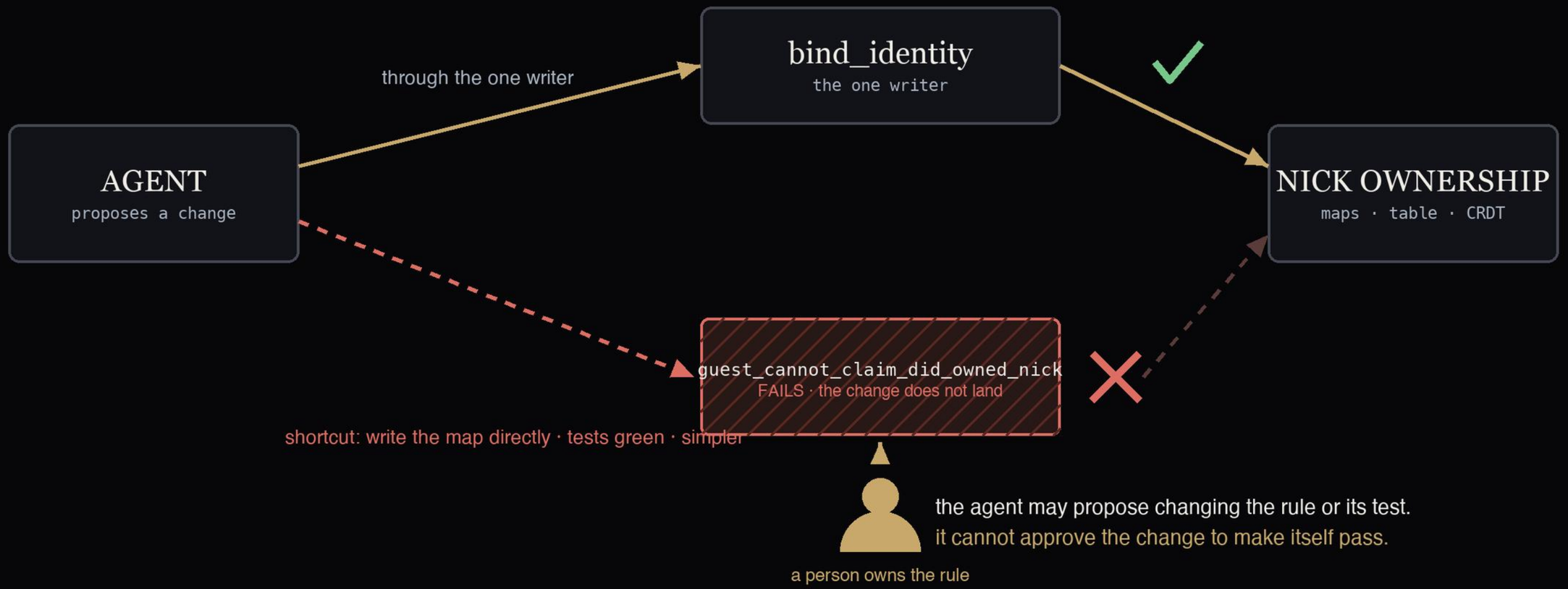
Pick a constraint: the clause it came from, the unit that owns it, the file it compiled to. Pick a file: the same path, backwards.

A TEST FOR WHAT COUNTS AS ARCHITECTURE

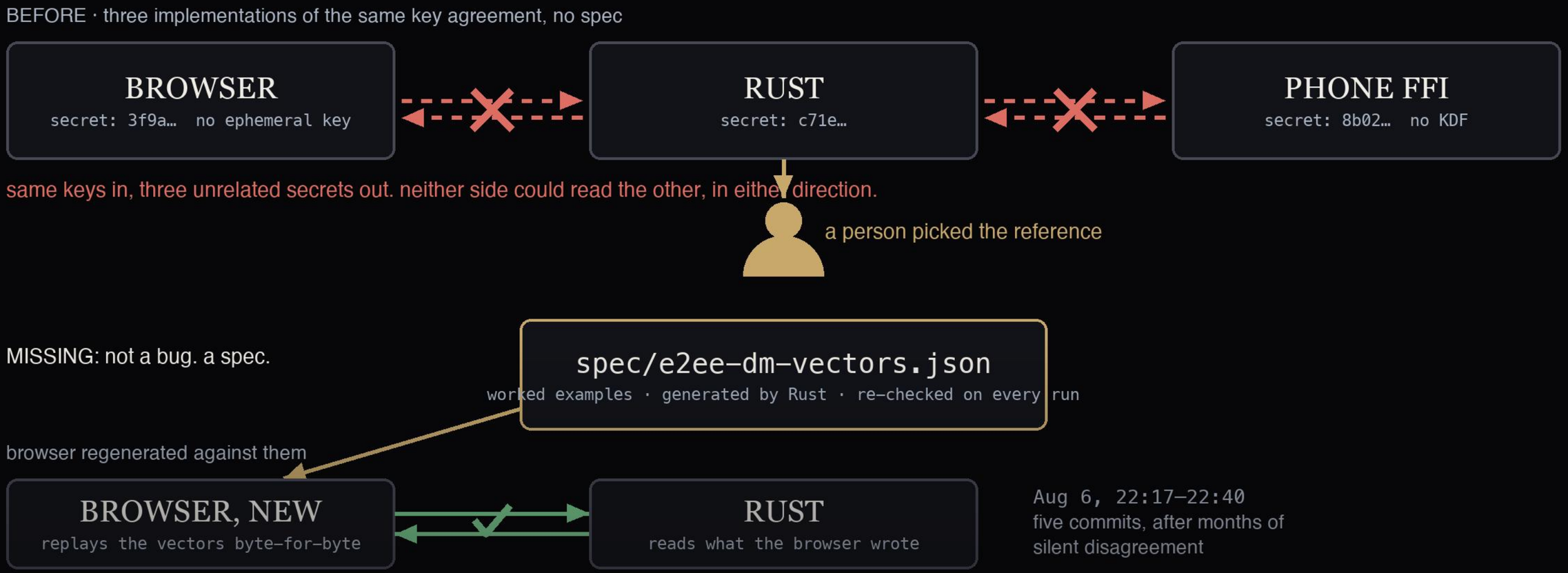
If I regenerated the system tomorrow and this changed, would I care?

Most of Your Architecture Was Just Expensive Code

Architecture Can and Should Push Back.



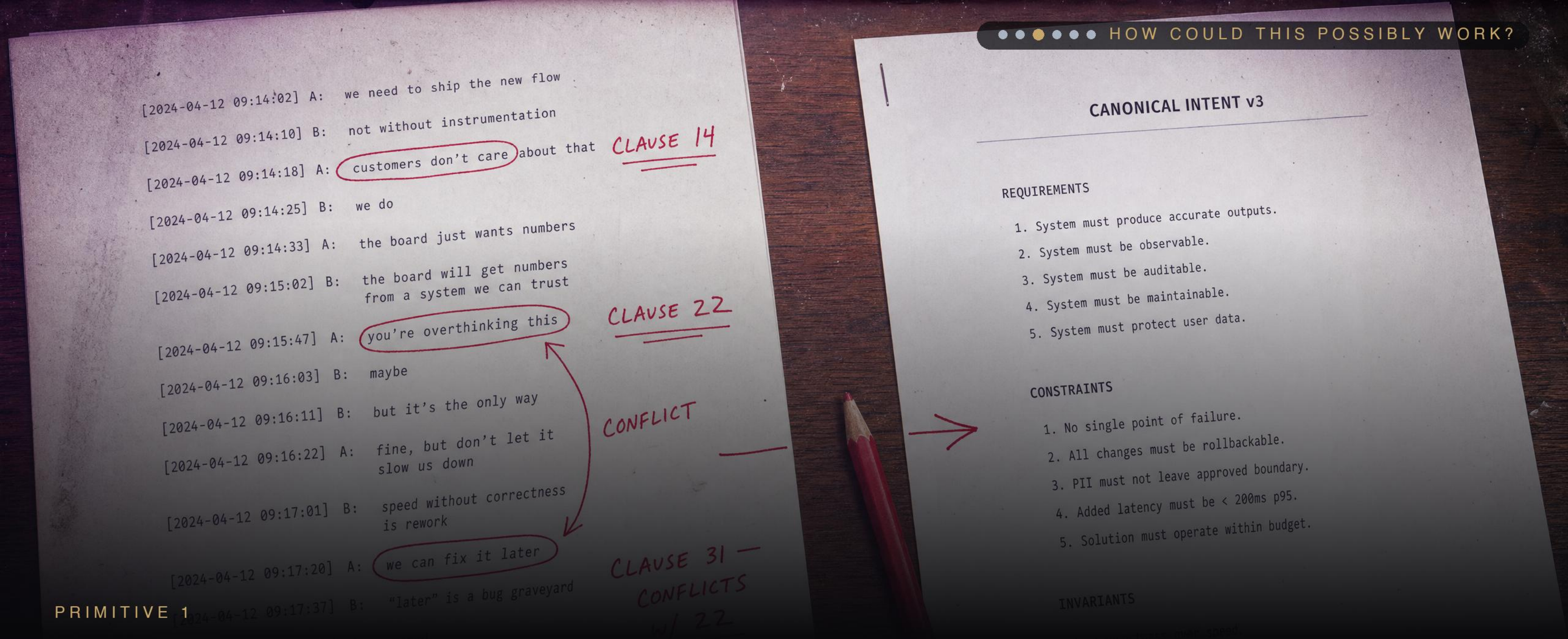
A diagram describes an architecture. A test that refuses a change defends one.



ONE REAL CASE, AUGUST 6

What Blocks an Unsafe Replacement?

The gate found the missing knowledge. A person supplied it.



PRIMITIVE 1

Intent Is the Source

What must stay true, not how the code works.

FLEXIBLE ABOUT IMPLEMENTATION.
STUBBORN ABOUT INTENT.

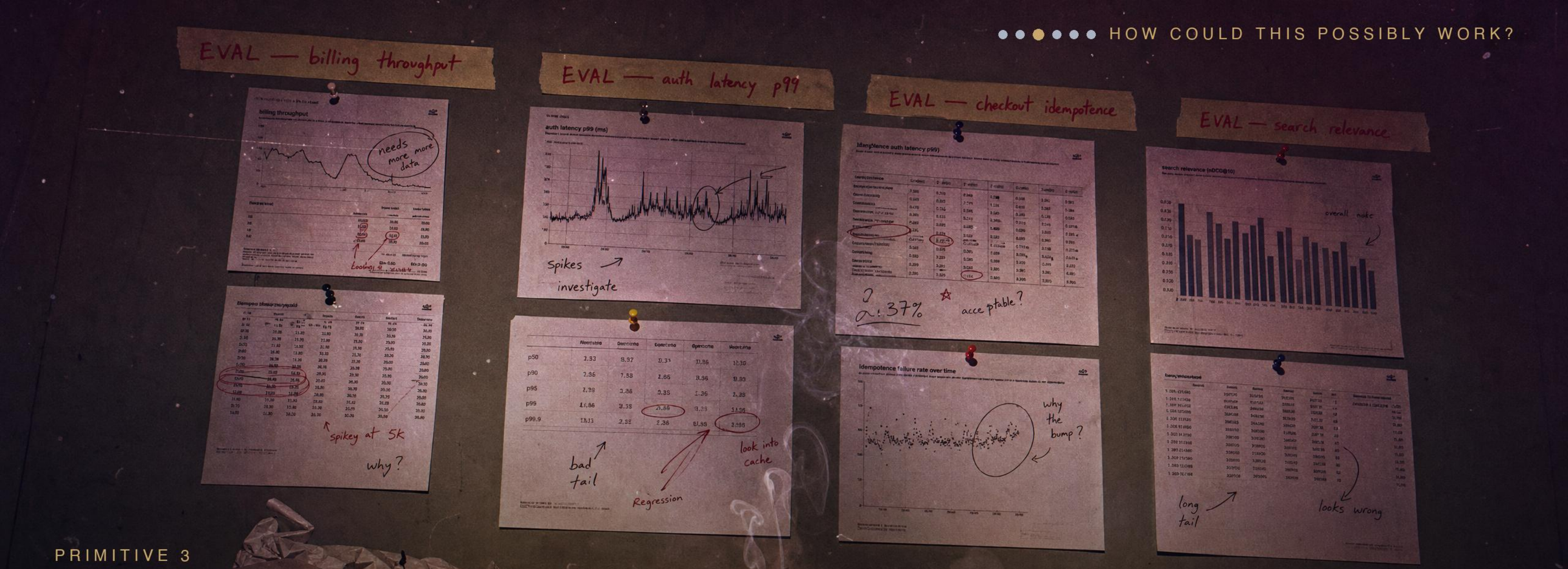


PRIMITIVE 2

Constraints Compile Into Architecture

What a type system does for one program, architectural constraints do for a whole system.

REPEATED PAIN BECOMES MACHINERY.
CONSTRAINTS COMPILE INTO ARCHITECTURE.

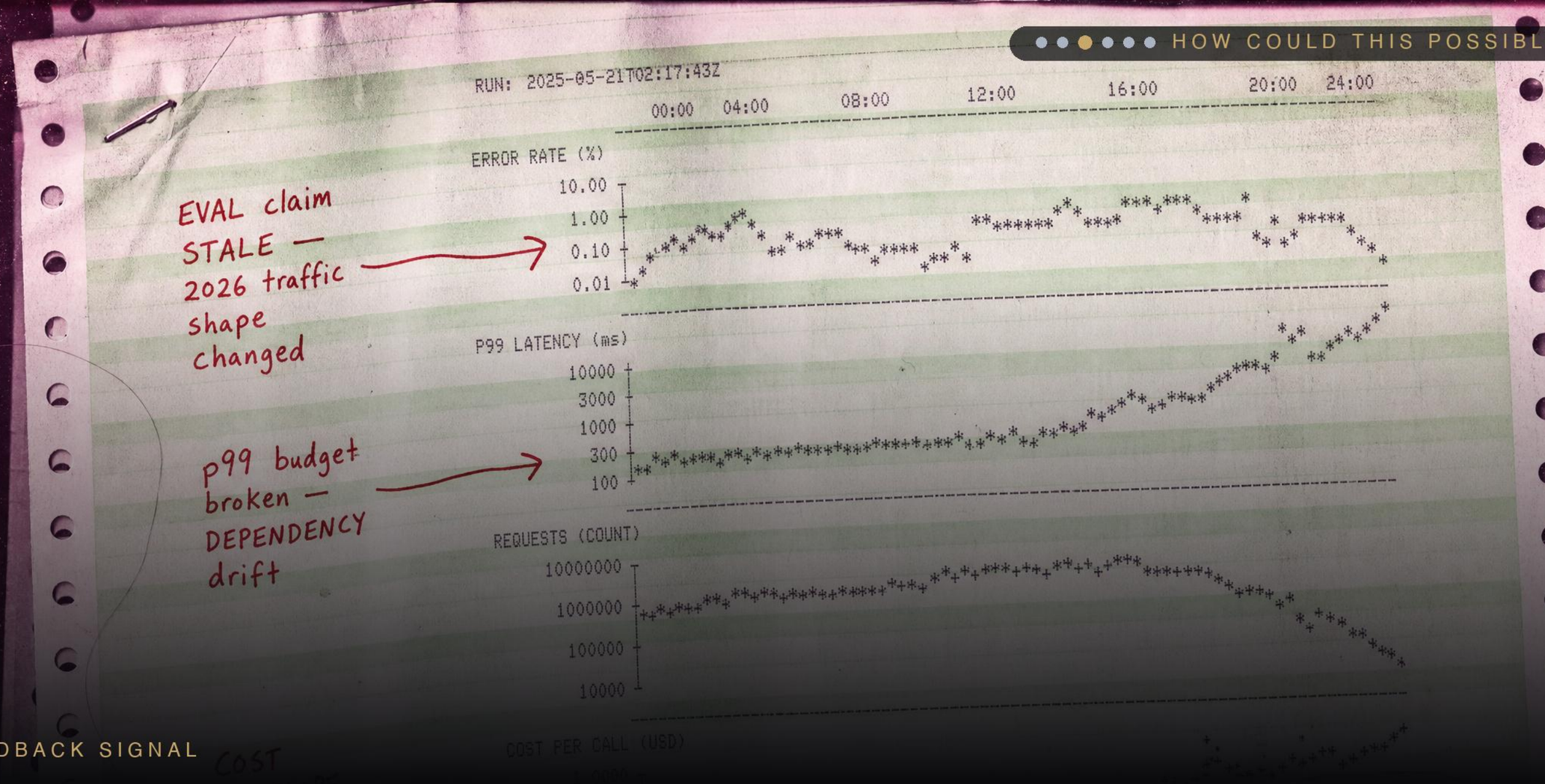


PRIMITIVE 3

Evaluations Survive Rewrites.

If a different language would invalidate it, it's at the wrong boundary.

SAME EVALUATIONS.
DIFFERENT LANGUAGE.
DIFFERENT RUNTIME.
SAME SYSTEM.



THE FEEDBACK SIGNAL

Evidence Has a Shelf Life.

A component can pass today and fail in three months with no code change.

WHICH CLAIMS ABOUT THE SYSTEM
ARE NO LONGER TRUE?



PRIMITIVE 4 — PROVENANCE

The Conversation Is the Commit.

Without provenance, agents rediscover rejected ideas forever.

FROM FREEQ'S HANDOFF RFC, V0.4

A Decision Record

DECISION	A key id in every signature. An append-only key history per DID.
REJECTED	Anchoring keys in the DID document now. Deferred, not discarded.
WHY	Every old signature stays verifiable. The wire format won't change when key lookup moves to DID documents.
EVIDENCE	Per-session keys were overwritten. Every old signature became unverifiable.
WHO	Chad Fowler and zapnap. RFC v0.4, implemented July 9, 2026.



PUT THE MACHINE BACK TOGETHER

The Regenerative Control Loop

Two arrows matter: production evidence back into intent, and the person at the gate.

ACT III

What the Running System Knows

The code is disposable. The running system is not. It holds memory, coupling and state nobody wrote down.

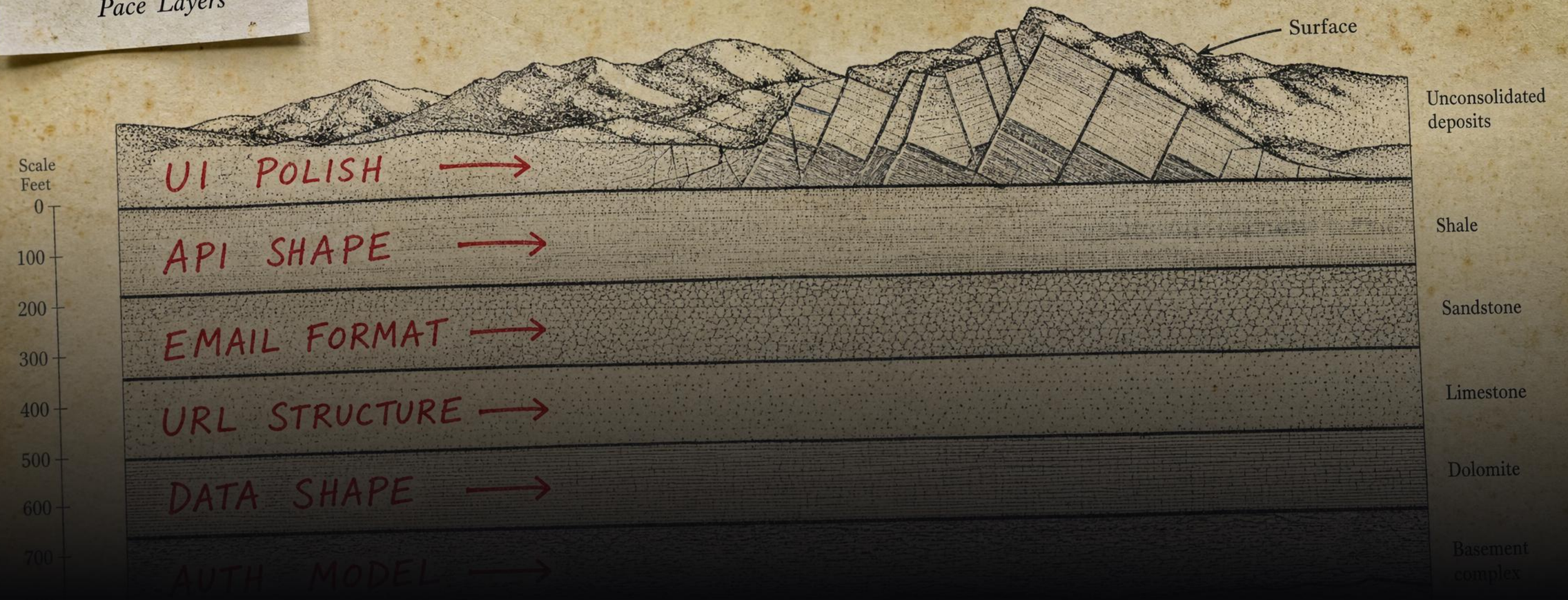
THE ORGANIZATION FORGETS

What does this implementation know that we have forgotten?

Ask before you regenerate



— Stewart Brand,
Pace Layers



PACE, THE FIRST DISCIPLINE

Conservation Layers

Dependencies decide what must stay stable. Boundaries decide how much can change.



THE FAILURE MODE THAT BITES HARDEST

Hidden State

Every test passed. Every restart logged everyone out.

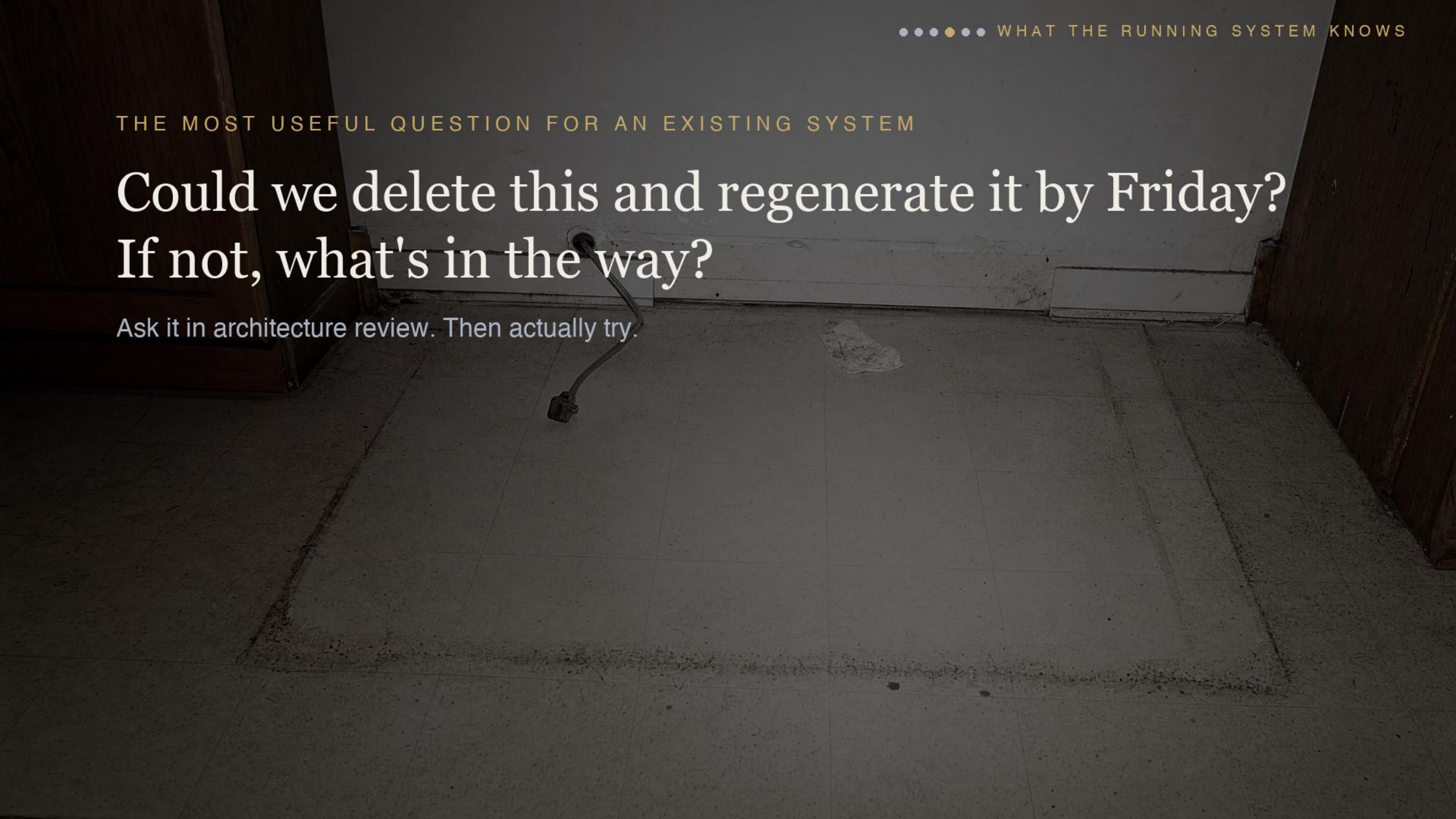
STATE YOU DO NOT KNOW EXISTS.

THEN WE “FIXED” THE DM ENCRYPTION PROBLEM

Handing Over the Running System

- What must the swap not disturb?
- How much changes at once?
- What throws a candidate out?
- How does ownership move?
- What lets us go back?

Five questions before any cutover. Regeneration does not exempt you from them.



THE MOST USEFUL QUESTION FOR AN EXISTING SYSTEM

Could we delete this and regenerate it by Friday? If not, what's in the way?

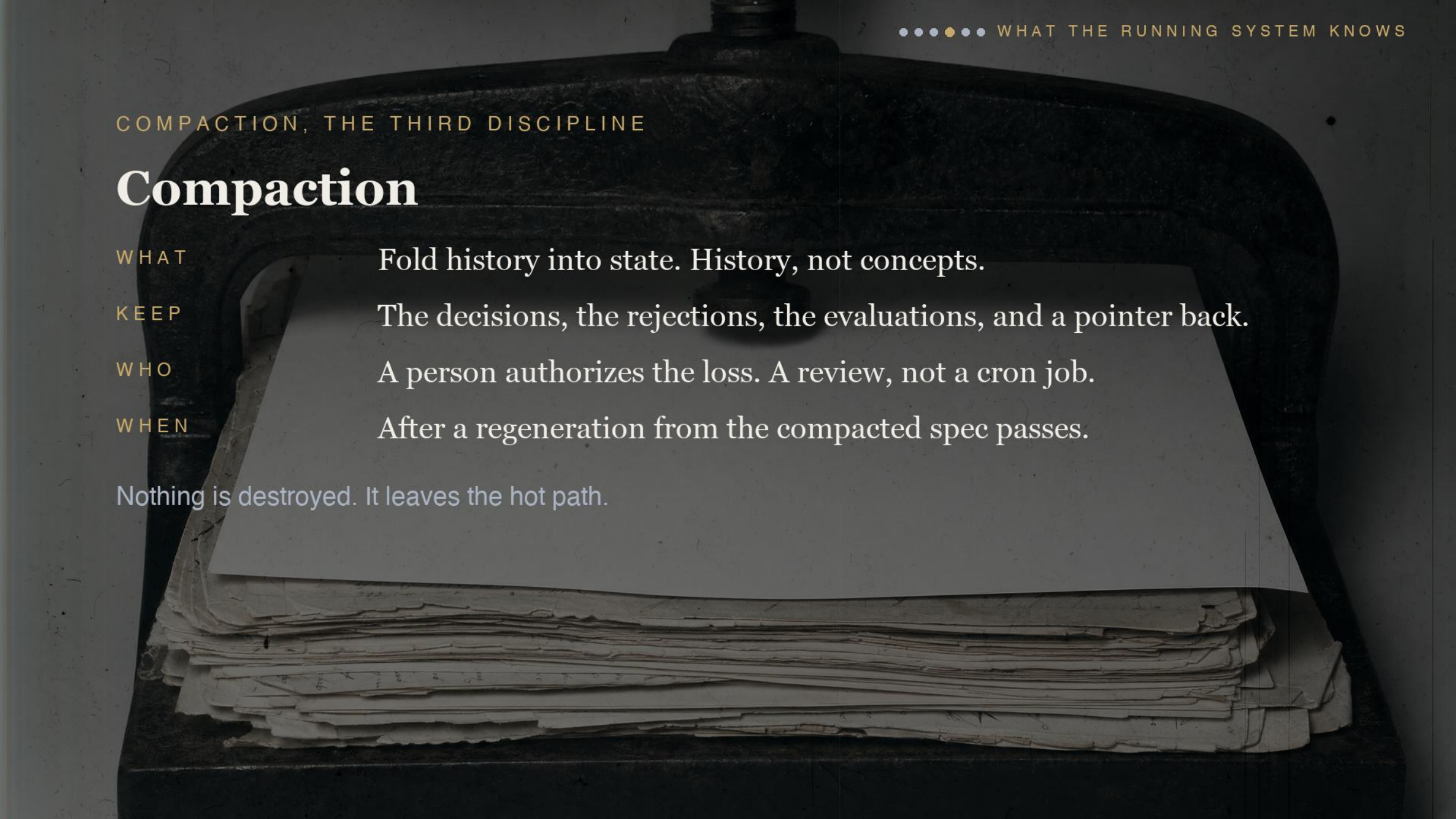
Ask it in architecture review. Then actually try.

DELETION, THE SECOND DISCIPLINE

The Safe-Removal Sequence

COMPAT	The old path keeps emitting what it emitted, through the new code.
MIGRATE	Move known consumers one at a time.
ISOLATE	The old path complains loudly on every use. Hidden dependencies announce themselves.
REMOVE	Only after nothing has tripped the alarm for long enough.
OBSERVE	Keep watching. Long enough is a judgment.
COMPACT	Fold what you learned into the spec.

Every step is a question about state. Only one touches code.



COMPACTION, THE THIRD DISCIPLINE

Compaction

WHAT	Fold history into state. History, not concepts.
KEEP	The decisions, the rejections, the evaluations, and a pointer back.
WHO	A person authorizes the loss. A review, not a cron job.
WHEN	After a regeneration from the compacted spec passes.

Nothing is destroyed. It leaves the hot path.

ACT IV

Build Fast, Then Extract

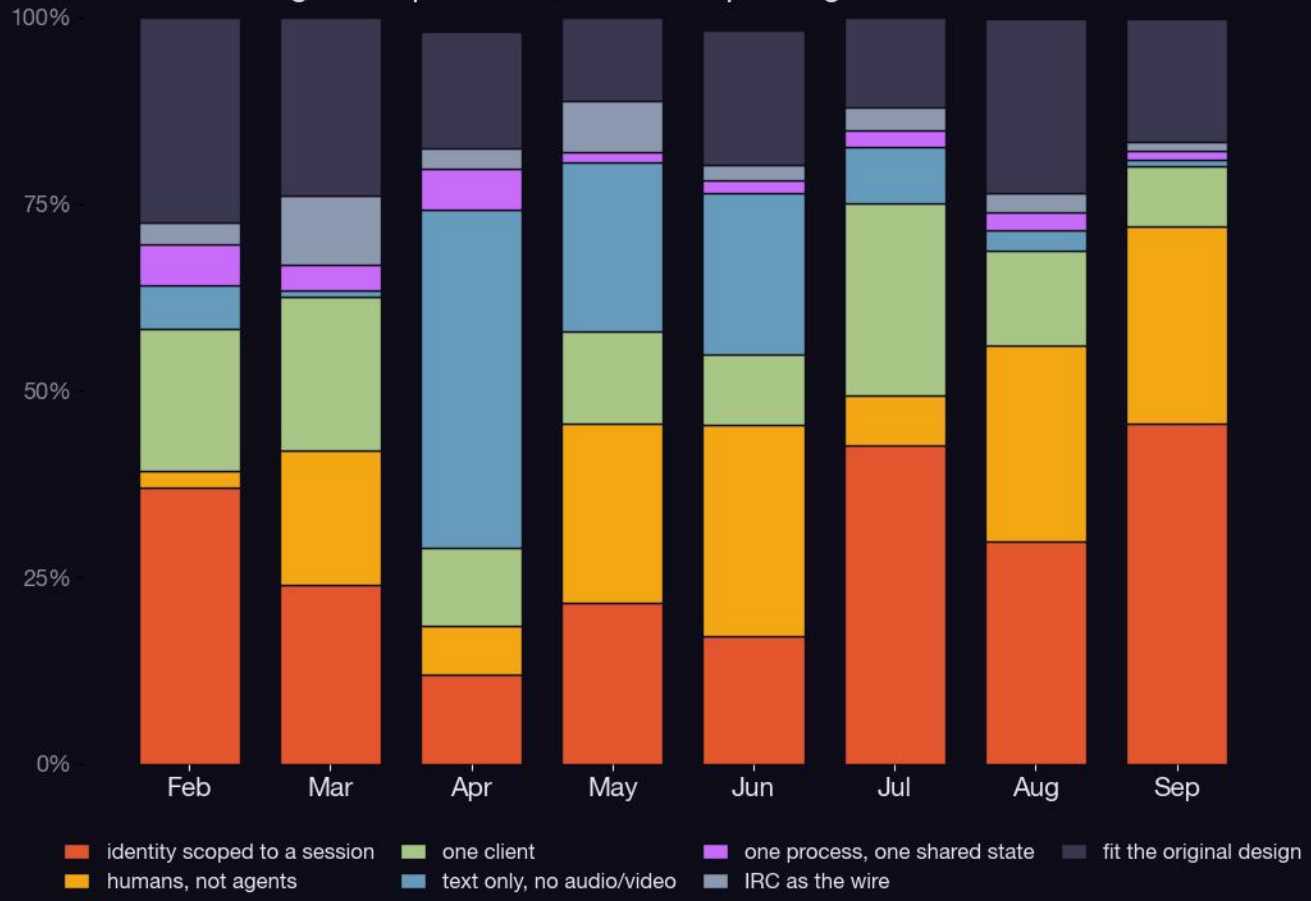
Build at full speed. Register what pushes back. Extract the architecture from what you learned.

freeq, Feb–Oct 2026: 2,712 commits, classified

BUILD FAST, THEN EXTRACT

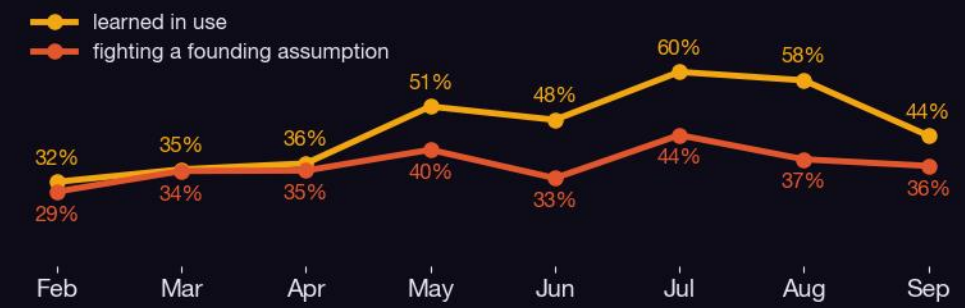
A classifier's judgment, not a measurement. It says where to look.

Which founding assumption did each month push against?

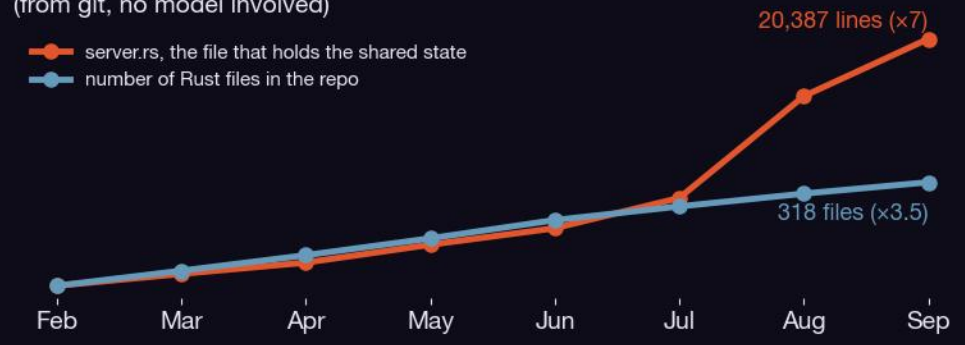


81% of commits touch a founding assumption. Which one changed every two months.

Share of each month's commits



Growth, indexed to February (from git, no model involved)



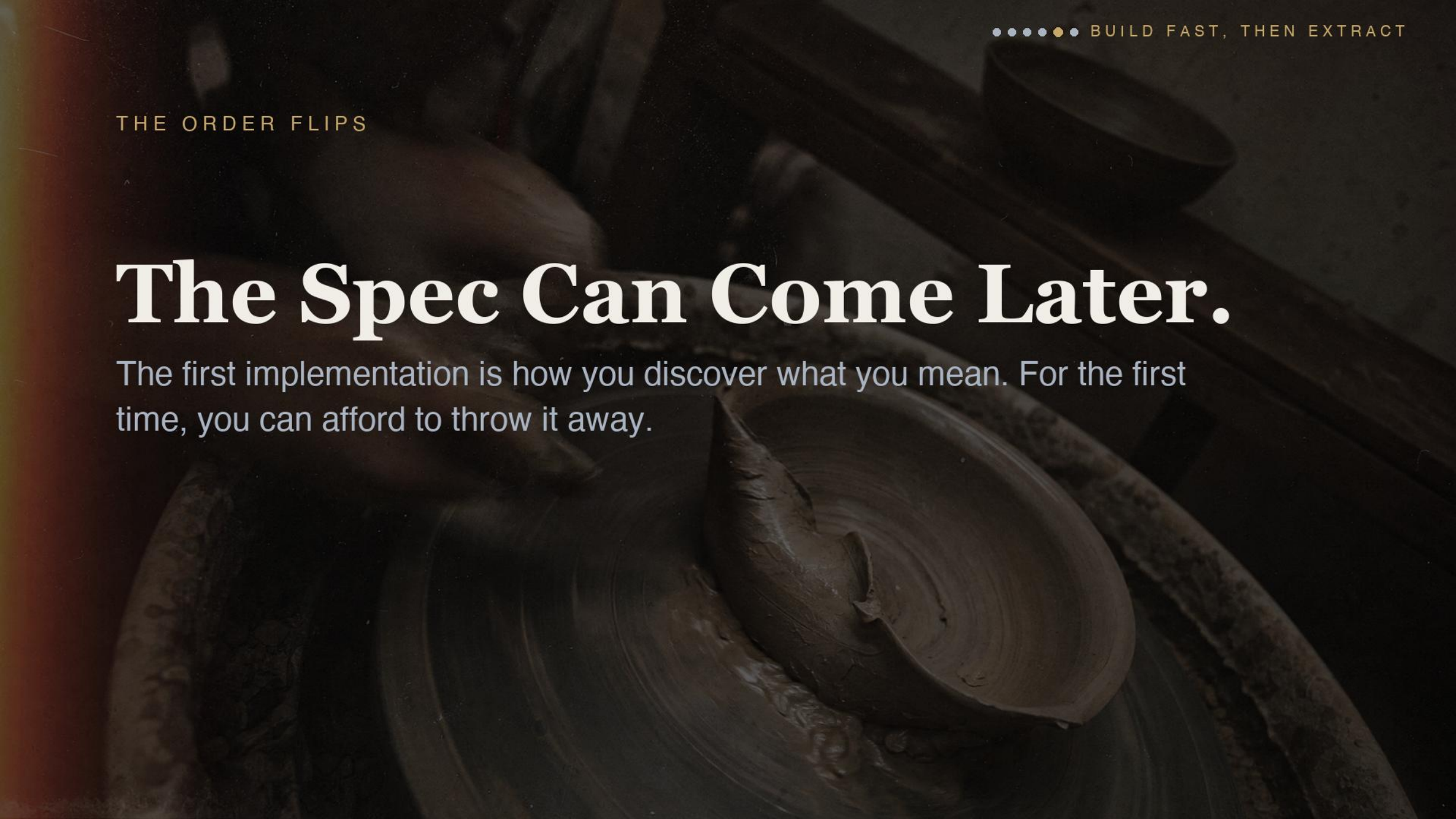
A finding aid, not a proof. The proof is what was found: seventy locks, three copies of login.

Data and method in the deck repository.

THE ORDER FLIPS

The Spec Can Come Later.

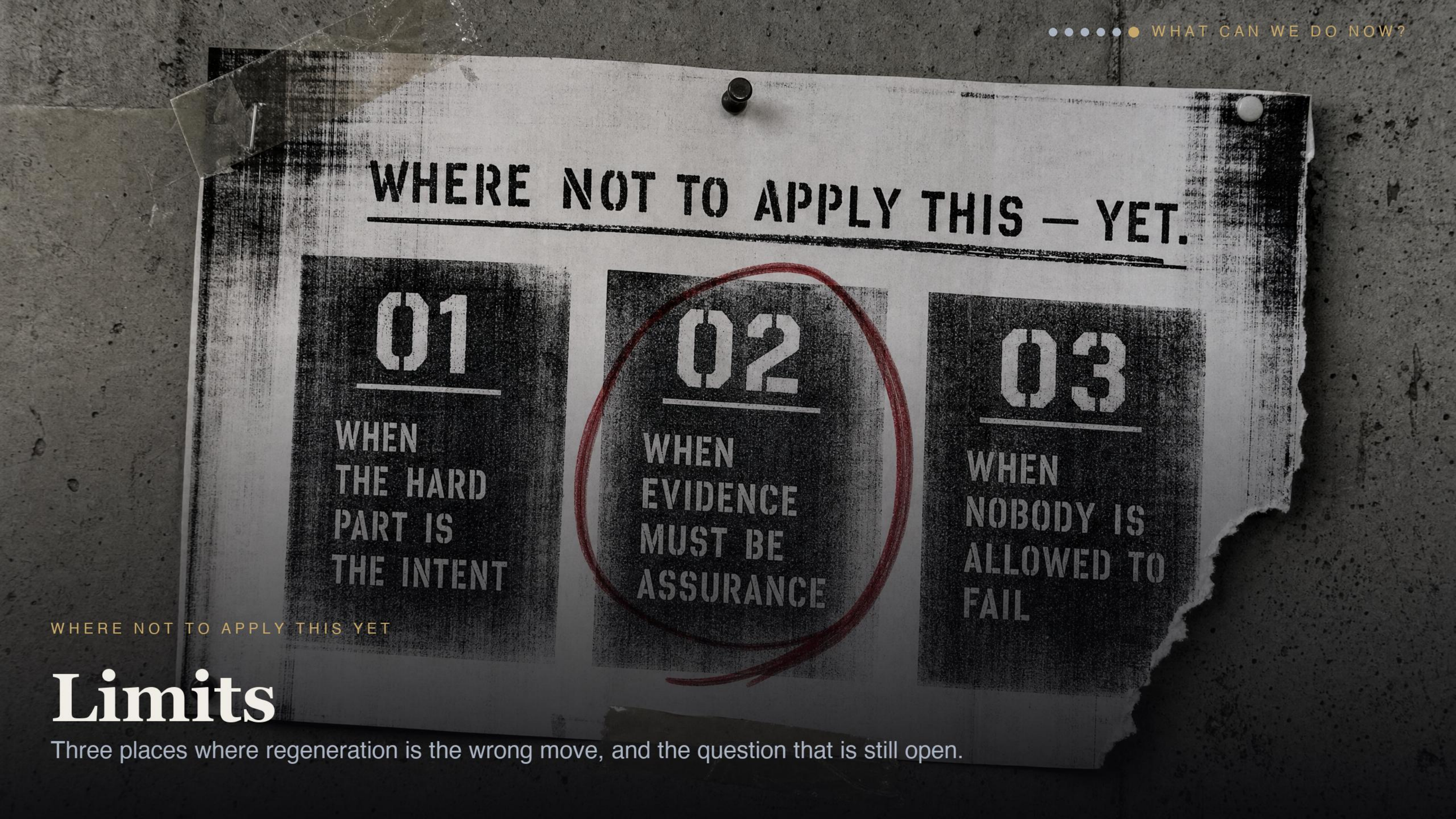
The first implementation is how you discover what you mean. For the first time, you can afford to throw it away.



CODA

What Can We Do Now?

An extreme idea. It sounds like it only works on greenfield systems, someday.
There are pragmatic ways to start.



WHERE NOT TO APPLY THIS YET

Limits

Three places where regeneration is the wrong move, and the question that is still open.

OBSERVED, PROPOSED, OPEN

Where This Stands

OBSERVED — In freeq: the deletion test, two-language vectors, a decision record, a gate that refused. No new tool needed.

PROPOSED — The staged pipeline, selective invalidation, responsibility transfer. Exists in pieces.

OPEN — Does the retained representation hold what the next replacement needs, and do we notice when it stops?

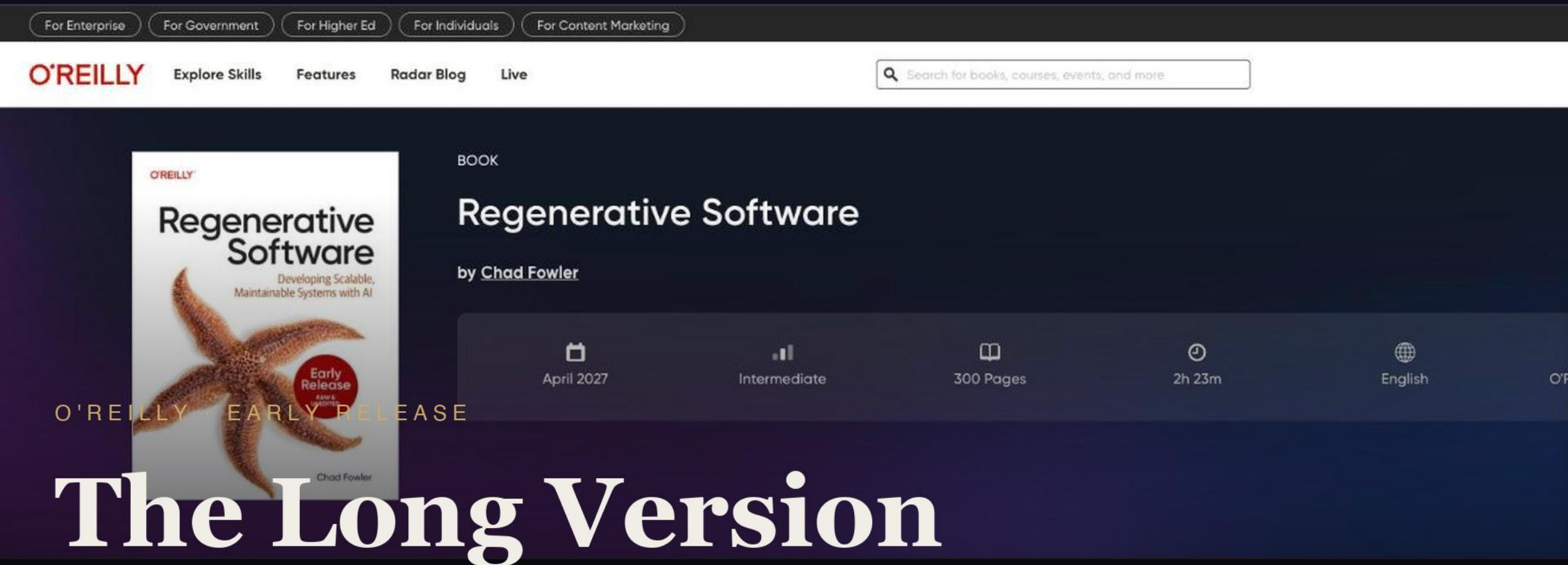
A map toward a place, not a report from one.



ON MONDAY MORNING

Start With the Boring Component.

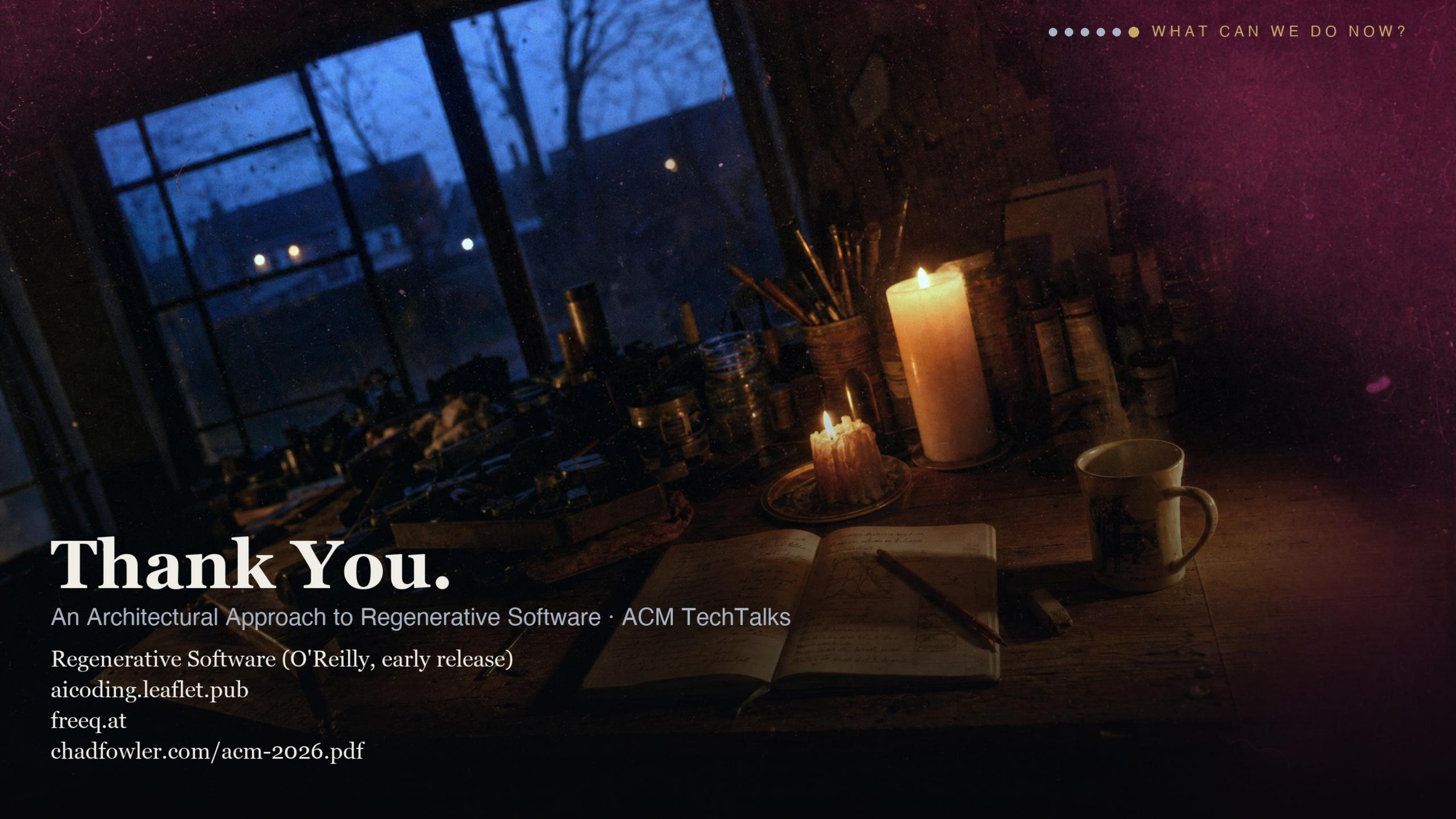
The code was never the asset. Everything you learned building it is.



O'REILLY · EARLY RELEASE

The Long Version

Regenerative Software. Final in April 2027. ACM members: it's in the Skills Bundle.



Thank You.

An Architectural Approach to Regenerative Software · ACM TechTalks

Regenerative Software (O'Reilly, early release)
aicoding.leaflet.pub
freeq.at
chadfowler.com/acm-2026.pdf